

Lập Trình Python Nâng cao

Giới Thiệu

Trịnh Tấn Đạt

Đại Học Sài Gòn

trinhtandat@sgu.edu.vn

<http://sites.google.com/site/ttdat88>

Thông Tin Môn Học

- 03 tín chỉ: 60 tiết (LT: 30; TH: 30)
- Nội dung chính: 10 chương
 - (1) Giới thiệu ngôn ngữ Python,
 - (2) Higher-order functions
 - (3) Đối tượng dữ liệu tuần tự
 - (4) Quản lý biến trong Python
 - (5) Lập trình hướng đối tượng trong Python
 - (6) Hiển thị trực quan dữ liệu
 - (7) Làm việc với các dữ liệu cơ bản
 - (8) Làm việc với các thư viện quan trọng trong Khoa học dữ liệu
 - (9) Làm việc với tensor nhiều chiều
 - (10) Các kỹ thuật xử lý dữ liệu

Mục Đích Môn Học

- **Về kiến thức:**

- Nắm vững các kỹ thuật các kỹ thuật trong khoa học dữ liệu dựa trên NNLT Python.
- Định hướng cho sinh viên hoàn thành các dự án môn học, nâng cao kỹ năng lập trình.

- **Về kỹ năng:**

- Sinh viên sử dụng thành thạo NNLT Python để phân tích và trực quan hóa dữ liệu.

- **Về thái độ:**

- Rèn luyện thái độ và tư duy làm việc nghiêm túc, khoa học.

Tài Liệu Học Tập

- Các tài liệu tham khảo
 - Mark Lutz, Learning Python: Powerful Object-Oriented Programming. O'Reilly Media, Inc. 2013
 - Think Python: How to think like a computer scientist
 - Python Data Science Handbook - Essential tools for working with data
 - Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems
 - Mastering Python Data Visualization
- Các nguồn từ Internet
 - <http://scipy-lectures.org/>
 - Python tutorial: <https://www.programiz.com/python-programming/first-program>
 - Github
 - Kaggle
 - Colab
 - ...

Đánh giá

- Điểm quá trình (50%)
 - Điểm danh (10%)
 - Đồ án giữa kỳ/bài tập nhóm (project) (40%): SV báo cáo đồ án giữa kỳ (lý thuyết + demo)
- Thi kết thúc môn học (50%): đề đóng tự luận

Đồ án giữa kỳ

Sinh viên làm việc theo nhóm (tối đa 04 thành viên/nhóm).

- Tìm hiểu và xây dựng các ứng dụng minh họa
- Viết báo cáo
- Chương trình demo
- Ứng dụng 1. Các thư viện và mô hình Machine Learning
- Ứng dụng 2. Phân tích, trực quan hóa và khai phá dữ liệu
- Ứng dụng 3. Phân tích dữ liệu thời gian và dự báo
- Ứng dụng 4. Xử lý ngôn ngữ tự nhiên
- Ứng dụng 5. Xử lý ảnh và thị giác máy tính
- Ứng dụng 6. Deep Learning và Generative AI
- Ứng dụng 7. Dữ liệu lớn, IoT và dữ liệu thời gian thực
- Ứng dụng 8. Các ứng dụng Khoa học dữ liệu trong tài chính, môi trường, giáo dục, y tế, giao thông và thương mại điện tử

Giới thiệu ngôn ngữ python

- Python
 - ngôn ngữ thông dịch
 - ngôn ngữ lập trình hướng đối tượng.
 - hoàn toàn tạo kiểu động và sử dụng cơ chế cấp phát bộ nhớ tự động.
 - có cấu trúc dữ liệu cấp cao mạnh mẽ.
 - cú pháp lệnh là điểm cộng vô cùng lớn vì sự rõ ràng, dễ hiểu và cách gõ linh động.
 - thuận tiện cho người mới học lập trình.
- Có thể chạy trên nhiều hệ điều hành khác nhau : *Unix, Windows, Mac OS, Linux...*

Giới thiệu ngôn ngữ python

- <https://www.24h.com.vn/kham-pha-cong-nghe/top-5-cong-nghe-lap-trinh-giup-sinh-vien-it-vuot-bao-cach-mang-40-c675a989128.html>
- <https://www.tiobe.com/tiobe-index/>

May 2019	May 2018	Change	Programming Language	Ratings	Change
1	1		Java	16.005%	-0.38%
2	2		C	14.243%	+0.24%
3	3		C++	8.095%	+0.43%
4	4		Python	7.830%	+2.64%
5	6	▲	Visual Basic .NET	5.193%	+1.07%
6	5	▼	C#	3.984%	-0.42%
7	8	▲	JavaScript	2.690%	-0.23%
8	9	▲	SQL	2.555%	+0.57%
9	7	▼	PHP	2.489%	-0.83%
10	13	▲	Assembly language	1.816%	+0.82%
11	15	▲	Objective-C	1.626%	+0.69%
12	12		Delphi/Object Pascal	1.406%	+0.39%
13	18	▲	Perl	1.394%	+0.48%
14	16	▲	MATLAB	1.366%	+0.44%
15	10	▼	Ruby	1.343%	+0.16%

Giới thiệu ngôn ngữ python

- Ví dụ : một đoạn python script

```
import math
import os
from copy import deepcopy

import cv2
import numpy as np
from PIL import Image, ImageEnhance
from numpy import asarray, mean, array, argmax
from skimage.transform import radon

def distance(v1, v2):
    dist = np.sqrt(np.sum(np.square(np.subtract(v1, v2))))
    return dist

def box_area(box):
    return (int(box[2]) - int(box[0])) * (int(box[3]) - int(box[1]))
```

Giới thiệu ngôn ngữ python

- Được tạo ra bởi Guido Van Rossum (sinh năm 1956). Thiết kế bắt đầu vào cuối những năm 1980 và được phát hành lần đầu tiên vào tháng 2 năm 1991.
- Python kế thừa từ ngôn ngữ ABC
- Tại sao lại có tên là Python?
 - Rossum là fan của một chương trình hài cuối những năm 1970, và cái tên “Python” được lấy từ tên một phân trong đó **“Monty Python’s Flying Circus”**.
- Python 2 được giới thiệu năm 2000
 - Hỗ trợ unicode
 - Mã python 2 rất phổ biến
- Python 3 được phát hành năm 2008
 - Hiện đã có phiên bản 3.7.3
- Bản phân phối python Anaconda
 - hỗ trợ rất nhiều thư viện (numpy, scipy, matplotlib , sklearn)
 - dùng nhiều trong machine learning, data science, AI



Tại Sao Sử Dụng Python

Tính năng chính của Python (Ưu điểm)

- **Ngôn ngữ lập trình đơn giản, dễ học:** Python có cú pháp rất đơn giản, rõ ràng.
- **Miễn phí, mã nguồn mở:** Python có một cộng đồng rộng lớn, không ngừng cải thiện nó mỗi lần cập nhật.
- **Khả năng di chuyển:** Các chương trình Python có thể di chuyển từ nền tảng này sang nền tảng khác và chạy nó mà không có bất kỳ thay đổi nào.
- **Khả năng mở rộng và có thể nhúng:** Giả sử một ứng dụng đòi hỏi sự phức tạp rất lớn, bạn có thể dễ dàng kết hợp các phần code bằng C, C++ (**module ctypes**) vào code Python.
- **Ngôn ngữ thông dịch cấp cao:** Không giống như C/C++ (**ngôn ngữ biên dịch**), với Python, bạn không phải lo lắng những nhiệm vụ khó khăn như quản lý bộ nhớ, dọn dẹp những dữ liệu.
- **Thư viện tiêu chuẩn lớn để giải quyết những tác vụ phổ biến**
- **Hướng đối tượng:** Mọi thứ trong Python đều là hướng đối tượng

Hạn chế

Ngôn ngữ này có những mặt hạn chế nhất định như sau:

- Python không có các thuộc tính như: protected, private hay public, không có vòng lặp do...while và switch....case.
- Python mặc dù nhanh hơn so với PHP, nhưng lại không nhanh hơn so với C/C++, Java.

Sử Dụng Python Để Làm Gì

Python được dùng ở đâu?

- **Lập trình ứng dụng web:** Bạn có thể tạo web app có khả năng mở rộng (scalable). Các trang như Mozilla, Reddit, Instagram đều được viết bằng Python.
- **Khoa học và tính toán:** Có nhiều thư viện trong Python cho khoa học và tính toán số liệu, như SciPy và NumPy. Ngoài ra, Python còn được sử dụng nhiều trong machine learning, data mining và deep learning (scikit-learn, tensorflow , keras, pytorch, ...)
- **Tạo nguyên mẫu phần mềm:** Python là ngôn ngữ tuyệt vời để tạo những nguyên mẫu (bản chạy thử - prototype). Ví dụ, bạn có thể sử dụng Pygame (thư viện viết game) để tạo nguyên mẫu game trước. Nếu thích nguyên mẫu đó có thể dùng C++ để viết game thực sự.
- **Ngôn ngữ tốt để dạy lập trình:** Python được nhiều công ty, trường học sử dụng để dạy lập trình cho trẻ em và những người mới lần đầu học lập trình.

Python trong khoa học dữ liệu

- Python có cú pháp rõ ràng, dễ đọc và phù hợp cho người mới bắt đầu.
- Hệ sinh thái phổ biến:
 - NumPy: tính toán mảng số.
 - Pandas: dữ liệu bảng.
 - Matplotlib/Seaborn: trực quan hóa.
 - Scikit-learn: học máy.
 - PyTorch/TensorFlow: học sâu.
- Trong học phần này, Python là nền tảng trước khi đi đến dữ liệu, trực quan hóa và tensor.

Môi trường cài đặt

Cài Đặt

1. Cài đặt Python 3 + IDE Pycharm

2. Cài đặt Python 3 dùng bản phân phối Anaconda (RAM > 4GB)

- **Cài đặt IDE Pycharm**
- Anaconda hỗ trợ IDE Spyder
- Jupyter Notebook

3. Sử dụng Google Colab, Kaggle

4. Visualize code execution : <https://pythontutor.com/>

Các lựa chọn môi trường phổ biến

Môi trường	Phù hợp với	Điểm mạnh
Python IDLE	Người mới học	Có sẵn khi cài Python
VS Code	Viết dự án, quản lý file	Linh hoạt, nhiều extension
Jupyter Notebook	Phân tích dữ liệu	Chạy từng cell, hiển thị bảng/biểu đồ
Google Colab	Học và thực hành trực tuyến	Không cần cài đặt, hỗ trợ GPU
PyCharm	Dự án Python lớn	Debug và quản lý project tốt

Từ Khóa Trong Python

- Những từ chỉ dành riêng cho Python
- Trong Python, ngoại trừ **True**, **False** và **None** được viết hoa ra thì các keyword khác đều được viết dưới dạng chữ thường, đây là điều bắt buộc.

• False	• class	• finally	• is	• return
• None	• continue	• for	• lambda	• try
• True	• def	• from	• nonlocal	• while
• and	• del	• global	• not	• with
• as	• elif	• if	• or	• yield
• assert	• else	• import	• pass	
• break	• except	• in	• raise	

Định Danh Trong Python

- Định danh là tên được đặt cho các thực thể như class, function, biến,... trong Python giúp phân biệt thực thể này với thực thể khác.
- Quy tắc viết định danh trong Python:
 - Định danh có thể là sự kết hợp giữa các chữ cái viết thường (**từ a đến z**) hoặc viết hoa (**A đến Z**) hoặc các số (**từ 0 đến 9**) hoặc dấu gạch dưới (**_**). Định danh hợp lệ sẽ giống như thế này: `bien_1`, `tinh_tong_0_9`, `firstClass`.
 - Định danh **không thể bắt đầu bằng một chữ số**, ví dụ `1bien` là không hợp lệ, nhưng `bien1` thì đúng.
 - Định danh phải **khác các keyword**. Bạn thử nhập `and = 1` rồi chạy sẽ xuất hiện thông báo lỗi `"SyntaxError: invalid syntax"`.
 - Python **không hỗ trợ các ký tự đặc biệt** như `!`, `@`, `#`, `$`, `%`,... trong định danh. Nếu cố tình gán các ký tự này trong định danh sẽ nhận được thông báo lỗi `"SyntaxError: invalid syntax"` hoặc `"NameError:..."`.
 - Python là ngôn ngữ lập trình **phân biệt chữ hoa, chữ thường**, nghĩa là `bien` và `Bien` là không giống nhau.

Biến trong Python

- Một biến Python là một khu vực bộ nhớ được dành riêng để lưu trữ các giá trị.
- Mỗi giá trị trong Python có một kiểu dữ liệu. Các kiểu dữ liệu khác nhau trong Python là
 - kiểu số (number)
 - kiểu danh sách (list)
 - kiểu bộ (tuple)
 - kiểu chuỗi (string)
 - kiểu từ điển (dictionary)
 - kiểu tập hợp (set)

Biến trong Python

- Biến trong python:
 - Có tên, phân biệt chữ hoa/thường
 - Không cần khai báo trước
 - Không cần chỉ ra kiểu dữ liệu
 - Có thể thay đổi sang kiểu dữ liệu khác
 - Nên gán giá trị ngay khi bắt đầu xuất hiện
 - Không được trùng với từ khóa
- **Cách khai báo và sử dụng biến**
 - Ví dụ: **n = 12** # biến n là kiểu nguyên
n = n + 0.1 # biến n chuyển sang kiểu thực
print(n)

Biến trong Python

Biến toàn cục và biến cục bộ

- Trong Python khi muốn sử dụng **cùng một biến cho toàn bộ chương trình hoặc mô-đun**, cần khai báo nó dưới dạng biến toàn cục.
- Trường hợp muốn sử dụng **biến trong một hàm hoặc phương thức cụ thể**, hãy sử dụng biến cục bộ.

Biến trong Python

```
# Declare a variable and initialize it
f = 101
print(f)
# Global vs. local variables in functions
def someFunction():
    f = 'I am learning Python'
    print(f)
someFunction()
print(f)
```

The screenshot shows a Python IDE window titled 'Python5.2.py'. The code is as follows:

```
1 # Declare a variable and initialize it
2 f = 101
3 print(f)
4
5 # Global vs. local variables in functions
6 def someFunction():
7     # global f
8     f = 'I am learning Python'
9     print(f)
10
11 someFunction()
12 print(f)
13
```

Annotations on the code:

- A red circle with the number '1' is next to the line `f = 101`.
- A red circle with the number '2' is next to the line `f = 'I am learning Python'` inside the function, which is also enclosed in a red rectangular box.
- A red circle with the number '3' is next to the line `print(f)` at the end of the script.

A dashed green line connects these three points in a loop, indicating the flow of the variable `f`. An orange callout box points to the line `f = 'I am learning Python'` with the text: `f` là biến cục bộ được khai báo bên trong hàm.

The output console at the bottom shows the following text:

```
"C:\Users\DK\Desktop\Python code\Python Test\Python 5\PythonCode5\PythonCode5\Python5.2.py"
101
I am learning Python
101
```

Arrows from the red circles point to the corresponding output lines: '1' points to the first '101', '2' points to 'I am learning Python', and '3' points to the second '101'.

Biến trong Python

global f

```
1 f = 101;
2 print(f) ①
3
4 # Global vs. local variables in functions
5 def someFunction():
6     global f ②
7     print(f)
8     f = "changing global variable"
9
10 someFunction()
11 print(f) ③
12
13
14
```

someFunction()

Run Python5.3

"C:\Users\DK\Desktop\Python code\Python Test\Python 5\PythonCode5\Python5.3.py"

101

101

changing global variable

Giờ chúng ta đang truy xuất và thay đổi giá trị của biến toàn cục f

Toán Tử Trong Python

- Python hỗ trợ nhiều phép toán số, logic, gán, so sánh và phép toán bit
- Toán tử số học:** $+$, $-$, $*$, $/$, $\%$, $**$ ($\%$: phép chia lấy dư ; $**$: lũy thừa)
Ví dụ : $2**3 = 8$; $3**3 = 27$
- Python có 2 phép chia:
 - chia đúng ($/$): $10/3$ # 3.3333333333333335
 - chia nguyên ($//$): $10/3$ # 3 (nhanh hơn phép $/$)
- Các phép so sánh:** $<$, $<=$, $>$, $>=$, $!=$, $==$

```
In [2]: runfile('C:/Users/sony/Desktop/
python_co_ban/test/Test1.py', wdir='C:/Users/
sony/Desktop/python_co_ban/test')
x > y is False
```

```
x = 4
y = 5
print('x > y is', x>y)
```

Toán Tử Trong Python

- **Toán tử gán:** Các toán tử gán khác nhau được sử dụng trong Python là (=, +=, -=, *=, /=, ...)

```
num1 = 4
num2 = 5

print("Line 1 - Value of num1 : ", num1)
print("Line 2 - Value of num2 : ", num2)
```

```
num1 = 4
num2 = 5

res = num1 + num2
res += num1

print("Line 1 - Result of + is ", res)
```

```
In [3]: runfile('C:/Users/sony/Desktop/
python_co_ban/test/Test1.py', wdir='C:/Users/
sony/Desktop/python_co_ban/test')
('Line 1 - Result of + is ', 13)
```

Toán Tử Trong Python

- Toán tử logic: **and, or, not**

```
a = True
b = False

print(('a and b is', a and b))
print(('a or b is', a or b))
print(('not a is', not a))
```

```
In [4]: runfile('C:/Users/son/python_co_ban/test/Test1.py'
sony/Desktop/python_co_ban/t
('a and b is', False)
('a or b is', True)
('not a is', False)
```

Toán Tử Trong Python

- **Toán tử thành viên** : Các toán tử này kiểm tra tư cách thành viên trong một tập như danh sách, chuỗi hoặc tuple. Có hai toán tử thành viên được sử dụng trong Python là (**in**, **not in**). Kết quả trả về phụ thuộc vào việc biến có tồn tại trong chuỗi hoặc tập cho trước hay không.

```
x = 4
y = 8
list = [1, 2, 3, 4, 5 ];
if ( x in list ):
    print("Line 1 - x is available in the given list")
else:
    print("Line 1 - x is not available in the given list")
if ( y not in list ):
    print("Line 2 - y is not available in the given list")
else:
    print("Line 2 - y is available in the given list")
```

```
In [5]: runfile('C:/Users/sony/Desktop/
python_co_ban/test/Test1.py', wdir='C:/Users/
sony/Desktop/python_co_ban/test')
Line 1 - x is available in the given list
Line 2 - y is not available in the given list
```

Cấu trúc rẽ nhánh if-else

```
if expression:
```

```
    # If-block
```

```
if expression:
```

```
    # If-block
```

```
elif 2-expression:
```

```
    # 2-if-block
```

```
elif 3-expression:
```

```
    # 3-if-block
```

```
...
```

```
elif n-expression:
```

```
    # n-if-block
```

```
if expression:
```

```
    # If-block
```

```
else:
```

```
    # else-block
```

```
if expression:
```

```
    # If-block
```

```
elif 2-expression:
```

```
    # 2-if-block
```

```
...
```

```
elif n-expression:
```

```
    # n-if-block
```

```
else:
```

```
    # else-block
```

Cấu trúc rẽ nhánh if-else

```
x = int(input("Nhap mot so: "))
if x < 0:
    x = 0
    print('So am')
elif x == 0:
    print('So 0')
elif x == 1:
    print('So 1')
else:
    print('So duong')
```

Vòng lặp while

```
while biểu_thức:  
    câu_lệnh
```

```
x=0  
#define a while loop  
while(x <4):  
    print(x)  
    x = x+1
```

Vòng lặp while

- Sử dụng while để tính tổng các số từ 1 đến n

```
n = int(input("Nhập n: ")) #Nhập số n tùy ý
tong = 0 #khởi báo và gán giá trị cho tong
i = 1 #khởi báo và gán giá trị cho biến đếm i

while i <= n:
    tong = tong + i
    i = i+1 # cập nhật biến đếm

print("Tổng là", tong)
```

Vòng lặp for

```
for variable_1, variable_2, .. variable_n in sequence:  
    # for-block
```

- Dùng hàm **range(a, b)** để tạo danh sách gồm các số từ **a** đến **b-1**, hoặc tổng quát hơn là **range(a, b, c)** trong đó **c** là bước nhảy

```
for d in range(10,20):           # in các số từ 10 đến 19  
    print(d)  
for d in range(20,10,-1):       # in các số từ 20 đến 11  
    print(d)
```

Vòng lặp for

- Ví dụ : Cách sử dụng vòng lặp for cho chuỗi

```
Months = ["Jan", "Feb", "Mar", "April", "May", "June"]  
for m in Months:  
    print(m)
```

Hàm enumerate

- Cách sử dụng hàm "enumerate" cho "vòng lặp For"
- Hàm enumerate trong "vòng lặp for" thực hiện hai điều:
 - Nó trả về giá trị chỉ số cho các phần tử
 - Và phần tử trong tập đang được xét.

```
#use a for loop over a collection  
Months = ["Jan", "Feb", "Mar", "April", "May", "June"]  
for i, m in enumerate (Months):  
    print(i,m)
```

Khi chương trình được thực thi, đầu ra của hàm enumerate trả về tên tháng với chỉ số của tháng trong danh sách như (0-Jan), (1-Feb), (2 – Mar), v.v.

Function

- Cú pháp khai báo hàm

```
def <tên-hàm>(danh-sách-tham-số):  
    <lệnh 1>  
    ...  
    <lệnh n>
```

```
def tinh(a, b):  
    return a*b
```

- Ví dụ: hàm tính tích 2 số

```
def tinh(a, b):  
    return a*b
```

Hàm trả về kết quả bằng lệnh **return**, nếu không trả về thì coi như trả về **None**

Function

- Hàm có thể chỉ ra giá trị mặc định của tham số
 - Chú ý: các tham số có giá trị mặc định phải đứng cuối danh sách tham số

```
def tinh(a, b = 1):  
    return a*b
```

```
print(tinh(10, 20))      # 200  
print(tinh(10))          # 10  
print(tinh(a=5))         # 5  
print(tinh(b=6, a=5))    # 30
```

Câu hỏi

Bài 1. Phân loại nhiệt độ

Viết chương trình nhập nhiệt độ theo độ C và in ra:

- < 20 : “Mát”
- $20-30$: “Bình thường”
- > 30 : “Nóng”

Bài 2. Tính chỉ số đơn giản

Cho danh sách:

$$\text{pm25} = [12.5, 18.2, 22.1, 15.4, 35.6]$$

Yêu cầu:

- Tính giá trị trung bình.
- Tìm giá trị lớn nhất.
- Đếm số ngày có PM_{2.5} lớn hơn 25.

Các kiểu dữ liệu nâng cao

Kiểu dữ liệu	Ký hiệu	Có thứ tự	Có thể thay đổi	Có phần tử trùng
List	[]	Có	Có	Có
Tuple	()	Có	Không	Có
Set	{ }	Không đảm bảo	Có	Không
Dictionary	{key: value}	Theo thứ tự chèn	Có	Key không trùng

Nội Dung

- Cách tạo List
- Các hàm và toán tử trên List
- Cách tạo 2D List
- Các hàm và toán tử trên 2D List

Tạo List

- **List Rỗng:** [] hoặc dùng **list()**

```
print("Two standard ways to create an empty list:")  
a = []  
b = list()  
print(type(a), len(a), a)  
print(type(b), len(b), b) print(a == b)
```

```
Two standard ways to create an empty list:  
<class 'list'> 0 []  
<class 'list'> 0 []  
True
```

Tạo List

- List có một phần tử

```
a = [ "hello" ]
```

```
b = [ 42 ]
```

```
print(type(a), len(a), a)
```

```
print(type(b), len(b), b)
```

```
print(a == b)
```

```
<class 'list'> 1 ['hello']
```

```
<class 'list'> 1 [42]
```

```
False
```

Tạo List

- **List có nhiều phần tử:** các phần tử cách nhau bởi dấu phẩy

```
a = [2, 3, 5, 7]
b = list(range(5))
c = ["mixed types", True, 42]
print(type(a), len(a), a)
print(type(b), len(b), b)
print(type(c), len(c), c)
```

```
<class 'list'> 4 [2, 3, 5, 7]
<class 'list'> 5 [0, 1, 2, 3, 4]
<class 'list'> 3 ['mixed types', True, 42]
```

Tạo List

```
n = 10  
a = [0] * n  
b = list(range(n))  
print(type(a), len(a), a)  
print(type(b), len(b), b)
```

```
<class 'list'> 10 [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]  
<class 'list'> 10 [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Các Hàm và Toán Tử Thông Dụng

- `len()`, `min()`, `max()`, `sum()`

```
a = [ 2, 3, 5, 2 ]  
print("a = ", a)  
print("len =", len(a))  
print("min =", min(a))  
print("max =", max(a))  
print("sum =", sum(a))
```

```
a = [2, 3, 5, 2]  
len = 4  
min = 2  
max = 5  
sum = 12
```

Các Hàm và Toán Tử Thông Dụng

- **Chỉ số trong List** : dùng toán tử []

```
a = [2, 3, 5, 7, 11, 13]
print("a =", a)
print("a[0] =", a[0])
print("a[2] =", a[2])
# Chỉ số âm
print("a[-1] =", a[-1])
print("a[-3] =", a[-3])
# một khoảng nằm trong list
print("a[0:2] =", a[0:2])
print("a[1:4] =", a[1:4])
print("a[1:6:2] =", a[1:6:2])
```

```
a = [2, 3, 5, 7, 11, 13]
a[0] = 2
a[2] = 5
a[-1] = 13
a[-3] = 7
a[0:2] = [2, 3]
a[1:4] = [3, 5, 7]
a[1:6:2] = [3, 7, 13]
```

Các Hàm và Toán Tử Thông Dụng

- **List Aliases:** Alias là khả năng mà tại 1 ô nhớ có nhiều đối tượng cùng

```
# Tao một list a
a = [ 2, 3, 5, 7 ]
# Tao một bi danh den list a
b = a
# Co hai tham chiếu cùng một list
a[0] = 42
b[1] = 99
print(a)
print(b)
```

```
[42, 99, 5, 7]
[42, 99, 5, 7]
```

```
a = [ 2, 3, 5, 7 ]
b = a
c = [ 2, 3, 5, 7 ]
print("initially:")
print(" a==b :", a==b)
print(" a==c :", a==c)
print(" a is b:", a is b)
print(" a is c:", a is c)
a[0] = 42
print("After changing a[0] to 42")
print(" a=",a)
print(" b=",b)
print(" c=",c)
print(" a==b :", a==b)
print(" a==c :", a==c)
print(" a is b:", a is b)
print(" a is c:", a is c)
```

Thông Dụng

```
a==b : True
```

```
a==c : True
```

```
a is b: True
```

```
a is c: False
```

After changing a[0] to 42

```
a= [42, 3, 5, 7]
```

```
b= [42, 3, 5, 7]
```

```
c= [2, 3, 5, 7]
```

```
a==b : True
```

```
a==c : False
```

```
a is b: True
```

```
a is c: False
```

Các Hàm và Toán Tử Thông Dụng

- **Tìm phần tử trong list: in và not in**

```
a = [ 2, 3, 5, 2, 6, 2, 2, 7 ]  
print("a =", a)  
print("2 in a =", (2 in a))  
print("4 in a =", (4 in a))
```

```
a = [2, 3, 5, 2, 6, 2, 2, 7]  
2 in a = True  
4 in a = False
```

```
a = [ 2, 3, 5, 2, 6, 2, 2, 7 ]  
print("a =", a)  
print("2 not in a =", (2 not in a))  
print("4 not in a =", (4 not in a))
```

```
a = [2, 3, 5, 2, 6, 2, 2, 7]  
2 not in a = False  
4 not in a = True
```

Các Hàm và Toán Tử Thông Dụng

- **Đếm số lần xuất hiện: `list.count(item)`**

```
a = [ 2, 3, 5, 2, 6, 2, 2, 7 ]  
print("a =", a)  
print("a.count(1) =", a.count(1))  
print("a.count(2) =", a.count(2))  
print("a.count(3) =", a.count(3))
```

```
a = [2, 3, 5, 2, 6, 2, 2, 7]  
a.count(1) = 0  
a.count(2) = 4  
a.count(3) = 1
```

Các Hàm và Toán Tử Thông Dụng

- Tìm chỉ số của một phần tử:

`list.index(item)` và `list.index(item, start)`

```
a = [ 2, 3, 5, 2, 6, 2, 2, 7 ]
print("a =", a)
print("a.index(6) =", a.index(6))
print("a.index(2) =", a.index(2))
print("a.index(2,1) =", a.index(2,1))
print("a.index(2,4) =", a.index(2,4))
print("a.index(2,7) =", a.index(2,7))
```

```
a = [2, 3, 5, 2, 6, 2, 2, 7]
a.index(6) = 4
a.index(2) = 0
a.index(2,1) = 3
a.index(2,4) = 5
Traceback (most recent call last):
ValueError: 2 is not in list
```

Các Hàm và Toán Tử Thông Dụng

- ❖ **Thêm phần tử hoặc một list vào list:** khi thêm sẽ thay đổi list hoặc tạo list mới.
- **Thêm một phần tử dùng: `list.append(item)`**

```
a = [ 2, 3 ]  
a.append(7)  
print(a)
```

```
[2, 3, 7]
```

- **Thêm một list vào một list: `list += list2`**

```
a = [ 2, 3 ]  
a += [ 11, 13 ]  
a += [0]  
print(a)
```

```
[2, 3, 11, 13, 0]
```

Các Hàm và Toán Tử Thông Dụng

- Thêm một list dùng `list.extend(list2)`

```
a = [ 2, 3 ]  
a.extend([ 17, 19 ])  
print(a)
```

`[2, 3, 17, 19]`

- Thêm một phần tử tại vị trí cho trước: dùng `insert()`

```
a = [ 2, 3, 5, 7, 11 ]  
a.insert(2, 42) # at index 2, insert 42  
print(a)
```

`[2, 3, 42, 5, 7, 11]`

Các Hàm và Toán Tử Thông Dụng

- Thêm phần tử hoặc list bằng cách tạo list mới.

```
a = [ 2, 3 ]  
b = a + [ 13, 17 ]  
print(a)  
print(b)
```

```
[2, 3]  
[2, 3, 13, 17]
```

```
a = [ 2, 3 ]  
b = a[:2] + [5] + a[2:]  
print(a)  
print(b)
```

```
[2, 3]  
[2, 3, 5]
```

```
print("Destructive:")  
a = [ 2, 3 ]  
b = a  
a += [ 4 ]  
print(a)  
print(b)  
print("Non-Destructive:")  
a = [ 2, 3 ]  
b = a  
a = a + [ 4 ]  
print(a)  
print(b)
```

Destructive:

[2, 3, 4]

[2, 3, 4]

Non-Destructive:

[2, 3, 4]

[2, 3]

Các Hàm và Toán Tử Thông Dụng

- Thay đổi các phần tử trong list

```
letters = ['a', 'b', 'c', 'd', 'e', 'f', 'g']
```

```
print(letters)
```

```
# thay the
```

```
letters[2:5] = ['C', 'D', 'E']
```

```
print(letters)
```

```
# xoa
```

```
letters[2:5] = []
```

```
print(letters)
```

```
# xóa list bằng cách thay tất cả các phần tử bằng một list rỗng
```

```
letters[:] = []
```

```
print(letters)
```

```
['a', 'b', 'c', 'd', 'e', 'f', 'g']  
['a', 'b', 'C', 'D', 'E', 'f', 'g']  
['a', 'b', 'f', 'g']  
[]
```

Các Hàm và Toán Tử Thông Dụng

- ❖ Xóa phần tử.
 - Xóa **một phần tử** dùng **list.remove(item)**

```
a = [ 2, 3, 5, 3, 7, 6, 5, 11, 13 ]  
print("a =", a)  
a.remove(5)  
print("After a.remove(5), a=", a)  
a.remove(5)  
print("After another a.remove(5), a=", a)
```

```
a = [2, 3, 5, 3, 7, 6, 5, 11, 13]  
After a.remove(5), a= [2, 3, 3, 7, 6, 5, 11, 13]  
After another a.remove(5), a= [2, 3, 3, 7, 6, 11, 13]
```

Các Hàm và Toán Tử Thông Dụng

- Xóa một phần tử dùng chỉ số : `list.pop(index)`

```
a = [ 2, 3, 4, 5, 6, 7, 8 ]
print("a =", a)
item = a.pop(3)
print("After item = a.pop(3)")
print(" item =", item)
print(" a =", a)
item = a.pop(3)
print("After another item = a.pop(3)")
print(" item =", item)
print(" a =", a)
# Xóa phần tử cuối cùng của list dùng list.pop()
item = a.pop()
print("After item = a.pop()")
print(" item =", item)

print(" a =", a)
```

```
a = [2, 3, 4, 5, 6, 7, 8]
After item = a.pop(3)
    item = 5
    a = [2, 3, 4, 6, 7, 8]
After another item = a.pop(3)
    item = 6
    a = [2, 3, 4, 7, 8]
After item = a.pop()
    item = 8
    a = [2, 3, 4, 7]
```

Các Hàm và Toán Tử Thông Dụng

- Xóa phần tử dùng toán tử del

```
a = [ 2, 3, 4, 5, 6, 7, 8 ]  
del a[2:4]  
print("a =", a)
```

```
a = [2, 3, 6, 7, 8]
```

- Phương thức clear() cũng được dùng để làm rỗng một list

```
a = [ 2, 3, 4, 5, 6, 7, 8 ]  
a.clear()  
print("a =", a)
```

```
a = []
```

Các Hàm và Toán Tử Thông Dụng

- Xóa phần tử dựa vào chỉ số bằng cách tạo list mới

```
a = [ 2, 3, 5, 3, 7, 5, 11, 13 ]  
print("a =", a)  
b = a[:2] + a[3:]  
print("After b = a[:2] + a[3:]")  
print(" a =", a)  
print(" b =", b)
```

```
a = [2, 3, 5, 3, 7, 5, 11, 13]  
After b = a[:2] + a[3:]  
    a = [2, 3, 5, 3, 7, 5, 11, 13]  
    b = [2, 3, 3, 7, 5, 11, 13]
```

Các Hàm và Toán Tử Thông Dụng

❖ Hoán đổi các phần tử (swapping)

- **Failed swap**

```
a = [ 2, 3, 5, 7 ]
```

```
print("a =", a)
```

```
a[0] = a[1]
```

```
a[1] = a[0]
```

```
print("After failed swap of a[0] and a[1]:")
```

```
print(" a=",a)
```

```
a = [2, 3, 5, 7]
```

```
After failed swap of a[0] and a[1]:
```

```
a= [3, 3, 5, 7]
```

Các Hàm và Toán Tử Thông Dụng

- **Swap dùng biến temp**

```
a = [ 2, 3, 5, 7 ]
print("a =", a)
temp = a[0]
a[0] = a[1]
a[1] = temp
print("After swapping a[0] and a[1]:")
print(" a=",a)
```

```
a = [2, 3, 5, 7]
After swapping a[0] and a[1]:
a= [3, 2, 5, 7]
```

Các Hàm và Toán Tử Thông Dụng

- Swap dùng **parallel assignment**

```
a = [ 2, 3, 5, 7 ]  
print("a =", a)  
a[0],a[1] = a[1],a[0]  
print("After swapping a[0] and a[1]:")  
print(" a=",a)
```

```
a = [2, 3, 5, 7]  
After swapping a[0] and a[1]:  
a= [3, 2, 5, 7]
```

Các Hàm và Toán Tử Thông Dụng

- ❖ Vòng lặp for trong list
 - Vòng lặp **dùng phần tử** trong list: **for item in list**

```
a = [ 2, 3, 5, 7 ]  
print("Here are the items in a:")  
for pt in a:  
    print(pt)
```

```
Here are the items in a:  
2  
3  
5  
7
```

Các Hàm và Toán Tử Thông Dụng

- Vòng lặp dùng chỉ số trong list : **for index in range(len(list))**

```
a = [ 2, 3, 5, 7 ]  
print("Here are the items in a with their indexes:")  
for i in range(len(a)):  
    print("a[", i, "] =", a[i])
```

```
Here are the items in a with their indexes:  
a[ 0 ] = 2  
a[ 1 ] = 3  
a[ 2 ] = 5  
a[ 3 ] = 7
```

Các Hàm và Toán Tử Thông Dụng

- **Duyệt ngược list dùng chỉ số**

```
a = [ 2, 3, 5, 7 ]
print("And here are the items in reverse:")
for index in range(len(a)):
    revIndex = len(a)-1-index
    print("a[", revIndex, "] =", a[revIndex])
```

```
And here are the items in reverse:
a[ 3 ] = 7
a[ 2 ] = 5
a[ 1 ] = 3
a[ 0 ] = 2
```

- **Duyệt ngược dùng hàm reversed()**

```
a = [ 2, 3, 5, 7 ]
print("And here are the items in reverse:")
for item in reversed(a):
    print(item)
print(a)
```

```
And here are the items in reverse:
7
5
3
2
[2, 3, 5, 7]
```

Các Hàm và Toán Tử Thông Dụng

- So sánh lists

```
# Create some Lists
a = [ 2, 3, 5, 3, 7 ]
b = [ 2, 3, 5, 3, 7 ] # same as a
c = [ 2, 3, 5, 3, 8 ] # differs in last element
d = [ 2, 3, 5 ] # prefix of a
print("a =", a)
print("b =", b)
print("c =", c)
print("d =", d)
print("-----")
print("a == b", (a == b))
print("a == c", (a == c))
print("a != b", (a != b))
print("a != c", (a != c))
print("-----")
print("a < c", (a < c))
print("a < d", (a < d))
```

```
a = [2, 3, 5, 3, 7]
b = [2, 3, 5, 3, 7]
c = [2, 3, 5, 3, 8]
d = [2, 3, 5]

-----

a == b True
a == c False
a != b False
a != c True

-----

a < c True
a < d False
```

```
# Create some lists
```

```
a = [ 2, 3]
```

```
b = [ 2, 3]
```

```
c = [ 2, 4]
```

```
d = [ 2]
```

```
e = [ 3]
```

```
f = [ 2 , 3 , 5]
```

```
print("-----")
```

```
print("a < c", (a < c))
```

```
print("a < d", (a < d))
```

```
print("a <= b", (a <= b))
```

```
print("a < b", (a < b))
```

```
print("a < d", (a < d))
```

```
print("a < e", (a < e))
```

```
print("a < f", (a < f))
```

Thân Dụng

```
a < c True
```

```
a < d False
```

```
a <= b True
```

```
a < b False
```

```
a < d False
```

```
a < e True
```

```
a < f True
```

Các Hàm và Toán Tử Thông Dụng

- **Copying list vs. List Aliasses**

```
import copy
# Create a list
a = [ 2, 3 ]
# Try to copy it
b = a # Error! Not a copy, but an alias
c = copy.copy(a) # Ok
print("At first...")
print(" a =", a)
print(" b =", b)
print(" c =", c)
# Now modify a[0]
a[0] = 42
print("But after a[0] = 42")
print(" a =", a)
print(" b =", b)
print(" c =", c)
```

At first...

```
a = [2, 3]
```

```
b = [2, 3]
```

```
c = [2, 3]
```

But after a[0] = 42

```
a = [42, 3]
```

```
b = [42, 3]
```

```
c = [2, 3]
```

Các Hàm và Toán Tử Thông Dụng

- Các cách sao chép list

```
import copy
```

```
a = [2, 3]
```

```
b = copy.copy(a)
```

```
c = a[:]
```

```
d = a + [ ]
```

```
e = list(a)
```

```
f = sorted(a)
```

```
a[0] = 42
```

```
print(a, b, c, d, e, f)
```

```
[42, 3] [2, 3] [2, 3] [2, 3] [2, 3] [2, 3]
```

Các Hàm và Toán Tử Thông Dụng

❖ Sắp xếp trên list

- Sắp xếp và thay đổi list dùng `list.sort()` :

`iterable.sort(key=None, reverse=False)`

```
a = [ 7, 2, 5, 3, 5, 11, 7 ]
b = list(a) # copy of a
print("At first, a =", a)
a.sort()
print("After a.sort(), a =", a)
#####
print("At first, b =", b)
b.sort(reverse=True)
print("After b.sort(reverse=True), b =", b)
```

```
At first, a = [7, 2, 5, 3, 5, 11, 7]
After a.sort(), a = [2, 3, 5, 5, 7, 7, 11]
At first, b = [7, 2, 5, 3, 5, 11, 7]
After b.sort(reverse=True), b = [11, 7, 7, 5, 5, 3, 2]
```

Các Hàm và Toán Tử Thông Dụng

- Sắp xếp nhưng không thay đổi list (tạo ra list mới) dùng `sorted(list)`

```
sorted(iterable, key=None, reverse=False)

a = [ 7, 2, 5, 3, 5, 11, 7 ]
print("At first")
print(" a =", a)
b = sorted(a)
c = sorted(a, reverse=True)
print("After b = sorted(a)")
print(" a =", a)
print(" b =", b)
print(" c =", c)
```

At first

```
a = [7, 2, 5, 3, 5, 11, 7]
```

After b = sorted(a)

```
a = [7, 2, 5, 3, 5, 11, 7]
```

```
b = [2, 3, 5, 5, 7, 7, 11]
```

```
c = [11, 7, 7, 5, 5, 3, 2]
```

Các Hàm và Toán Tử Thông Dụng

❖ Sắp xếp với key function

- Dùng `abs()`

```
a = [ 10, 2, -5, 8, -3, 7, 1 ]  
print(sorted(a))  
print(sorted(a, key=abs))
```

```
[-5, -3, 1, 2, 7, 8, 10]  
[1, 2, -3, -5, 7, 8, 10]
```

Các Hàm và Toán Tử Thông Dụng

- Sort dựa vào chiều dài của chuỗi

```
a = ['a', 'ab', 'aab', 'ac', 'abccc']
```

```
print(sorted(a))
```

```
#####
```

```
def mylensort(a):  
    return len(a)
```

```
print(sorted(a, key=mylensort))
```

```
['a', 'aab', 'ab', 'abccc', 'ac']
```

```
['a', 'ac', 'ab', 'aab', 'abccc']
```

Các Hàm và Toán Tử Thông Dụng

- So sánh thời gian và không gian bộ nhớ sử dụng giữa `sort()` và `sorted()`

<https://viblo.asia/p/so-sanh-listsort-voi-sortedlist-trong-python-gDVK22MrKLj>

Các Hàm và Toán Tử Thông Dụng

- **List và Function:** list được dùng như input của một hàm

```
def countOdds(a):  
    count = 0  
    for item in a:  
        if (item % 2 == 1):  
            count += 1  
    return count  
  
print(countOdds([2, 3, 7, 8, 21, 23, 24]))
```

4

Các Hàm và Toán Tử Thông Dụng

- Dùng hàm để thay đổi các giá trị của list

```
def fill(a, value):  
    for i in range(len(a)):  
        a[i] = value  
  
a = [1, 2, 3, 4, 5]  
print("At first, a =", a)  
fill(a, 42)  
print("After fill(a, 42), a =", a)
```

At first, a = [1, 2, 3, 4, 5]

After fill(a, 42), a = [42, 42, 42, 42, 42]

Các Hàm và Toán Tử Thông Dụng

- **List comprehension (Cách tạo list mới ngắn gọn):** là một biểu thức đi kèm với lệnh for được đặt trong cặp dấu ngoặc vuông [].

```
cub3 = [3 ** x for x in range(9)]  
# Output: [1, 3, 9, 27, 81, 243, 729, 2187, 6561]  
print(cub3)
```

Code trên tương đương với:

```
cub3 = []  
for x in range (9):  
    cub3.append(3**x)  
print(cub3)
```

Các Hàm và Toán Tử Thông Dụng

```
cub3 = [3 ** x for x in range(9) if x > 4]
```

```
# Output: [243, 729, 2187, 6561]
```

```
print(cub3)
```

```
so_le = [x for x in range(18) if x % 2 == 1]
```

```
# Output: [1, 3, 5, 7, 9, 11, 13, 15, 17]
```

```
print(so_le)
```

```
noi_list = [x+y for x in ['Ngôn ngữ ', 'Lập trình '] for y in ['Python', 'C++']]
```

```
# Output: ['Ngôn ngữ Python', 'Ngôn ngữ C++', 'Lập trình Python', 'Lập trình C++']
```

```
print(noi_list)
```

Các Hàm và Toán Tử Thông Dụng

- **Converting Between Lists and Strings**

```
# use list(s) to convert a string to a list of characters
```

```
a = list("wahoo!")
```

```
print(a) # prints: ['w', 'a', 'h', 'o', 'o', '!']
```

```
a = "How are you doing today?".split(" ")
```

```
print(a) # prints ['How', 'are', 'you', 'doing', 'today?']
```

```
a = ["parsley", " ", "is", " ", "gharsley"]
```

```
s = "".join(a)
```

```
print(s) # prints: parsley is gharsley
```

Tạo 2D List

- **Cấp phát tĩnh**

```
# Tao 2D list voi cac gia tri co dinh  
a = [ [ 2, 3, 4 ] , [ 5, 6, 7 ] ]  
print(a)
```

```
[[2, 3, 4], [5, 6, 7]]
```

Tạo 2D List

❖ Cấp phát động

▪ a) Wrong: Cannot use *

```
rows = 3
cols = 2
a = [ [0] * cols ] * rows # Error:
# Chỉ tạo duy nhất một dòng (unique row), phần còn lại là aliases!
print("This SEEMS ok. At first:")
print(" a =", a)
a[0][0] = 42
print("But see what happens after a[0][0]=42")
print(" a =", a)
```

This SEEMS ok. At first:

```
a = [[0, 0], [0, 0], [0, 0]]
```

But see what happens after a[0][0]=42

```
a = [[42, 0], [42, 0], [42, 0]]
```

Tạo 2D List

❖ Cấp phát động

▪ b) Mở rộng mỗi dòng

```
rows = 3
cols = 2
a=[]
for row in range(rows):
    a += [[0]*cols]
print("This IS ok. At first:")
print(" a =", a)
a[0][0] = 42
print("And now see what happens after a[0][0]=42")
print(" a =", a)
```

This IS ok. At first:

```
a = [[0, 0], [0, 0], [0, 0]]
```

And now see what happens after a[0][0]=42

```
a = [[42, 0], [0, 0], [0, 0]]
```

Tạo 2D List

❖ Cấp phát động

▪ c) Dùng list comprehension

```
rows = 3
cols = 2
a = [ ([0] * cols) for row in range(rows) ]
print("This IS ok. At first:")
print(" a =", a)
a[0][0] = 42
print("And now see what happens after a[0][0]=42")
print(" a =", a)
```

This IS ok. At first:

```
a = [[0, 0], [0, 0], [0, 0]]
```

And now see what happens after a[0][0]=42

```
a = [[42, 0], [0, 0], [0, 0]]
```

Các hàm và toán tử trên 2D List

- **Tìm số chiều của 2D List: dùng `len()`**

```
a = [ [ 2, 3, 5] , [ 1, 4, 7 ] ]  
print("a = ", a)  
# Now find its dimensions  
rows = len(a)  
cols = len(a[0])  
print("rows =", rows)  
print("cols =", cols)
```

```
a = [[2, 3, 5], [1, 4, 7]]  
rows = 2  
cols = 3
```

Các hàm và toán tử trên 2D List

- **Vòng lặp**

```
a = [ [ 2, 3, 5] , [ 1, 4, 7 ] ]  
print("Before: a =", a)  
rows = len(a)  
cols = len(a[0])  
for row in range(rows):  
    for col in range(cols):  
        a[row][col] += 1  
print("After: a =", a)
```

```
Before: a = [[2, 3, 5], [1, 4, 7]]  
After:  a = [[3, 4, 6], [2, 5, 8]]
```

Các hàm và toán tử trên 2D List

- ❖ Truy cập 2D List bằng dòng hoặc cột
 - a) Truy cập một dòng

```
# alias (not a copy!); (no new list created)
a = [ [ 1, 2, 3 ] , [ 4, 5, 6 ] ]
row = 1
rowList = a[row]
print(rowList)
rowList[0] = 100
print(rowList)
print(a)
```

```
[4, 5, 6]
[100, 5, 6]
[[1, 2, 3], [100, 5, 6]]
```

Các hàm và toán tử trên 2D List

- ❖ Truy cập 2D List bằng dòng hoặc cột
 - b) Truy cập một cột

```
# copy (not an alias!); (new list created)
a = [ [ 1, 2, 3 ] , [ 4, 5, 6 ] ]
col = 1
colList = [ ]
for i in range(len(a)):
    colList += [ a[i][col] ]
print(colList)
```

[2, 5]

có thể dùng **list comprehension**

```
colList = [ a[i][col] for i in range(len(a)) ]
```

Các hàm và toán tử trên 2D List

- Số lượng phần tử mỗi dòng có thể khác nhau

```
# 2d lists do not have to be rectangular
```

```
a = [ [ 1, 2, 3 ] ,  
      [ 4, 5 ] ,  
      [ 6 ] ,  
      [ 7, 8, 9, 10 ] ]
```

```
rows = len(a)
```

```
for row in range(rows):
```

```
    cols = len(a[row]) # now cols depends on each row
```

```
    print("Row", row, "has", cols, "columns: ", end="")
```

```
    for col in range(cols):
```

```
        print(a[row][col], " ", end="")
```

```
    print()
```

```
Row 0 has 3 columns: 1 2 3
```

```
Row 1 has 2 columns: 4 5
```

```
Row 2 has 1 columns: 6
```

```
Row 3 has 4 columns: 7 8 9 10
```

Tuples

Nội Dung

- Cách tạo Tuple
- Các hàm và toán tử trên Tuple

Cách Tạo Tuple

- **Tuple** (hay còn gọi là danh sách bất biến - **Immutable List**): là một chuỗi các phần tử có thứ tự giống như list. Sự khác biệt giữa list và tuple là chúng ta **không thể thay đổi các phần tử trong tuple khi đã gán**, nhưng trong list thì các phần tử có thể thay đổi.

```
t = (1, 2, 3)
print(type(t), len(t), t)
a = [1, 2, 3]
t = tuple(a)
print(type(t), len(t), t)
```

```
<class 'tuple'> 3 (1, 2, 3)
<class 'tuple'> 3 (1, 2, 3)
```

Cách Tạo Tuple

- Không thể thay đổi giá trị trong tuple khi đã gán (tương tự như chuỗi)

```
t = (1, 2, 3)
print(t[0])
t[0] = 42 # crash!
print(t[0])
```

```
1
```

```
Traceback (most recent call last):
```

```
TypeError: 'tuple' object does not support item assignment
```

Cách Tạo Tuple

```
t = (10, "quan tri mang", 2j)
#t[0:2] = (10, 'quan tri mang')
print("t[0:2] = ", t[0:2])
# Tuple rỗng
# Output: ()
my_tuple = ()
print(my_tuple)
# tuple số nguyên
# Output: (2, 4, 16, 256)
my_tuple = (2, 4, 16, 256)
print(my_tuple)
```

```
# tuple có nhiều kiểu dữ liệu
# Output: (10, "Quantrimang.com", 3.5)
my_tuple = (10, "Quantrimang.com", 3.5)
print(my_tuple)
# tuple lồng nhau
# Output: ("QTM", [2, 4, 6], (3, 5, 7))
my_tuple = ("QTM", [2, 4, 6], (3, 5, 7))
print(my_tuple)
# tuple có thể được tạo mà không cần dấu ()
# còn gọi là đóng gói tuple
# Output: (10, "Quantrimang.com", 3.5)
my_tuple = 10, "Quantrimang.com", 3.5
print(my_tuple)
# mở gói (unpacking) tuple cũng có thể làm được
# Output:
# 10
# Quantrimang.com
# 3.5
a, b, c = my_tuple
print(a)
print(b)
print(c)
```

Cách Tạo Tuple

- Tuple có những lợi thế nhất định so với list:
 - Tuple chứa những phần tử không thay đổi, có thể được sử dụng như key cho dictionary. Với list, điều này không thể làm được.
 - Nếu có dữ liệu không thay đổi việc triển khai nó như một tuple sẽ đảm bảo rằng dữ liệu đó **được bảo vệ chống ghi** (write-protected).

Các Hàm và Toán Tử Trên Tuple

- Parallel (tuple) assignment

```
(x, y) = (1, 2)
```

```
print(x)
```

```
print(y)
```

```
(x, y) = (y, x)
```

```
print(x)
```

```
print(y)
```

1

2

2

1

Các Hàm và Toán Tử Trên Tuple

- Singleton tuple syntax : **Tạo tuple chỉ có một phần tử** hơi phức tạp chút, nếu tạo theo cách thông thường là cho phần tử đó vào trong cặp dấu () là chưa đủ, cần phải thêm dấu phẩy để chỉ ra

```
t = (42)
```

```
print(type(t), t*5)
```

```
t = (42,)
```

```
print(type(t), t*5)
```

```
t = (42,2)
```

```
print(type(t), t*2)
```

```
<class 'int'> 210
```

```
<class 'tuple'> (42, 42, 42, 42, 42)
```

```
<class 'tuple'> (42, 2, 42, 2, 42, 2, 42, 2, 42, 2)
```

Các Hàm và Toán Tử Trên Tuple

```
t = [42]
print(type(t), t)
t = (42)
print(type(t), t)
t = (42)
print(type(t), t*5)
t = (42,)
print(type(t), t*5)
t = [42]
print(type(t), t*5)
```

```
<class 'list'> [42]
```

```
<class 'int'> 42
```

```
<class 'int'> 210
```

```
<class 'tuple'> (42, 42, 42, 42, 42)
```

```
<class 'list'> [42, 42, 42, 42, 42]
```

Các Hàm và Toán Tử Trên Tuple

- Truy cập vào các phần tử của tuple (tương tự list): dùng index [n] hoặc [a:b] hoặc [-k]

```
# tuple lồng nhau
n_tuple = ("Quantrimang.com", [2, 6, 8], (1, 2, 3))
# index lồng nhau
# Output: 'r'
print(n_tuple[0][5])
# index lồng nhau
# Output: 8
print(n_tuple[1][2])
```

Các Hàm và Toán Tử Trên Tuple

- **Thay đổi một tuple:** Không giống như list, **tuple không thể thay đổi**.
 - Điều này có nghĩa là các phần tử của một tuple không thể thay đổi một khi đã được gán.
 - Nhưng, nếu **bản thân phần tử đó là một kiểu dữ liệu có thể thay đổi** (như list chẳng hạn) thì **các phần tử lồng nhau có thể được thay đổi**. Chúng ta cũng có thể gán giá trị khác cho tuple (gọi là gán lại - **reassignment**).

```
my_tuple = (1, 3, 5, [7, 9])
```

```
#không thể thay đổi phần tử của tuple
```

```
#Bạn sẽ nhận được lỗi:
```

```
#TypeError: 'tuple' object does not support item assignment
```

```
my_tuple[1] = 9
```

Các Hàm và Toán Tử Trên Tuple

```
my_tuple = (1, 3, 5, [7, 9])  
# Nhưng phần tử có index 3 trong tuple là list  
# list có thể thay đổi, nên phần tử đó có thể thay đổi  
# Output: (1, 3, 5, [8, 9])  
my_tuple[3][0] = 8  
print(my_tuple)
```

Các Hàm và Toán Tử Trên Tuple

```
my_tuple = (1, 3, 5, [7, 9])  
# Nếu cần thay đổi tuple hãy gán lại giá trị cho nó  
# Output: ('q', 'u', 'a', 'n', 't', 'r', 'i', 'm', 'a', 'n', 'g')  
my_tuple = ('q', 'u', 'a', 'n', 't', 'r', 'i', 'm', 'a', 'n', 'g')  
print(my_tuple)
```

Các Hàm và Toán Tử Trên Tuple

- Nối tuple (+) và lặp lại tuple (*)

```
# Nối 2 tuple
```

```
# Output: (2, 4, 6, 3, 5, 7)
```

```
print((2, 4, 6) + (3, 5, 7))
```

```
# Lặp lại tuple
```

```
# Output: ('Quantrimang.com', 'Quantrimang.com', 'Quantrimang.com')
```

```
print(("Quantrimang.com",) * 3)
```

Các Hàm và Toán Tử Trên Tuple

- **Xóa tuple:** Các phần tử trong tuple không thể thay đổi nên chúng ta cũng **không thể xóa, loại bỏ phần tử khỏi tuple**.
- Nhưng việc xóa hoàn toàn một tuple có thể thực hiện được với từ khóa `del` như dưới đây

```
QTM = ('q','u','a','n','t','r','i','m','a','n','g','.', 'c','o','m')
```

```
# Không thể xóa phần tử của tuple
```

```
# TypeError: 'tuple' object doesn't support item deletion
```

```
del QTM[3]
```

```
# Có thể xóa toàn bộ tuple
```

```
del QTM
```

Các Hàm và Toán Tử Trên Tuple

- **count(x)**: Đếm số phần tử x trong tuple.
- **index(x)**: Trả về giá trị index của phần tử x đầu tiên mà nó gặp trong tuple.

```
QTM = ('q','u','a','n','t','r','i','m','a','n','g','.', 'c','o','m')
```

```
# Count
```

```
# Output: 2
```

```
print(QTM.count('m'))
```

```
# Index
```

```
# Output: 3
```

```
print(QTM.index('n'))
```

Các Hàm và Toán Tử Trên Tuple

- Toán tử: `in` và `not in`

```
QTM = ('q','u','a','n','t','r','i','m','a','n','g','.', 'c','o','m')
```

```
# Kiểm tra phần tử
```

```
# Output: True
```

```
print('a' in QTM)
```

```
# Output: False
```

```
print('b' in QTM)
```

```
# Output: False
```

```
print('g' not in QTM)
```

Các Hàm và Toán Tử Trên Tuple

- Vòng lặp và tuple

```
for ngon_ngu in ('Python','C++','Web'):  
    print("Lập trình",ngon_ngu)
```


Nội Dung

- Cách tạo Set
- Các hàm và toán tử trên Set

Cách tạo Set

- Set trong Python là tập hợp các phần tử **duy nhất, không có thứ tự**.
- Các phần tử trong set phân cách nhau bằng dấu phẩy và nằm trong dấu ngoặc nhọn {} hoặc sử dụng hàm **set()**
- Các phần tử trong set có thể thay đổi, có thể **thêm** hoặc **xóa phần tử** của set

Cách tạo Set

- Ví dụ:

```
s= set([2,3,5])  
print(3 in s) # prints True  
print(4 in s) # prints False  
for x in range(7):  
    if (x not in s):  
        print(x) # prints 0 1 4 6
```

Cách tạo Set

- **Tạo set rỗng**

```
s = set()
print(s) # prints set()
```

- **Tạo set từ list**

```
s = set(["cat", "cow", "dog"])
print(s)
```

```
set({'cat', 'cow', 'dog'})
```

- **Tạo set từ các đối tượng lặp lại**

```
s = set("wahoo")
print(s)
```

```
set({'w', 'a', 'h', 'o'})
```

Cách tạo Set

- Ví dụ

```
s = { 2, 3, 5 }  
print(s)  
ss = {1,2,1,5,4,2,3,6,4,5}  
print(ss)
```

```
set({2, 3, 5})  
set({1, 2, 5, 4, 3, 6})
```

Cách tạo Set

- * Lưu ý: { } is not an empty set!

```
s = { }  
print(type(s) == set) # False!  
print(type(s)) # This is a dictionary
```

```
False  
<class 'dict'>
```

Các Hàm và Toán Tử Trên Set

- Các tính chất của Set:

a) Vì set là tập hợp các phần tử **không có thứ tự** nên chỉ mục chả có ý nghĩa gì với set. Do đó toán tử cắt [] sẽ không làm việc trên set.

```
s = set([2,4,8])
print(s)
for element in s:
    print(element)

print(s[0])    # Error
print(s[:3])   # Error
```

```
set({2, 4, 8})
2
4
8
Traceback (most recent call last):
AttributeError: 'set' object has no attribute '__getitem__'
```

Các Hàm và Toán Tử Trên Set

- Các tính chất của Set:

b) Các phần tử là duy nhất (không trùng nhau)

```
s = set([2, 2, 2])  
print(s)  
print(len(s))
```

```
set({2})  
1
```

Các Hàm và Toán Tử Trên Set

- Các tính chất của Set:

c) Set không thể chứa phần tử có thể thay đổi được như list hay

```
a = ["lists", "are", "mutable"]
s = [a]
print("s:", s)
s1 = set(a)
print("s1:", s1)
s2 = set([a]) ### Error không thể tạo set {"lists", "are", "mutable"}
print("s2:", s2)
```

```
s: [['lists', 'are', 'mutable']]
s1: set({'lists', 'are', 'mutable'})
Traceback (most recent call last):
TypeError: unhashable type: 'list'
```

Các Hàm và Toán Tử Trên Set

```
s1 = set(["sets", "are", "mutable", "too"])
print("s1:", s1)
s2 = [s1]
print("s2:", s2)
s3 = set(s1)
print("s3:", s3)
s4 = set([s1])    ### Error không thể tạo set([set({'sets', 'are', 'mutable', 'too'})])
```

```
s1: set({'sets', 'are', 'mutable', 'too'})
s2: [set({'sets', 'are', 'mutable', 'too'})]
s3: set({'sets', 'are', 'mutable', 'too'})
Traceback (most recent call last):
TypeError: unhashable type: 'set'
```

Các Hàm và Toán Tử Trên Set

- Set thì hiệu quả hơn list: Set dùng cơ chế hàm băm (hashing) để kiểm tra sự tồn tại của phần tử

0. Preliminaries

```
import time
```

```
n = 500
```

1. Create a list [2,4,6,...,n] then check for membership # among [1,2,3,...,n] in that list. # don't count the list creation in the timing

```
a = list(range(2,n+1,2))
```

```
print("Using a list... ", end="")
```

```
start = time.time()
```

```
count = 0
```

```
for x in range(n+1):
```

```
    if x in a:
```

```
        count += 1
```

```
end = time.time()
```

```
elapsed1 = end - start
```

```
print("count=", count, " and time = %.4f seconds" % elapsed1)
```

2. Repeat, using a set

```
print("Using a set.... ", end="")
```

```
start = time.time()
```

```
s = set(a) count = 0
```

```
for x in range(n+1):
```

```
    if x in s:
```

```
        count += 1
```

```
end = time.time()
```

```
elapsed2 = end - start
```

```
print("count=", count, " and time = %.4f seconds" % elapsed2)
```

```
print("With n=%d, sets ran about %.1f times faster than lists!" % (n, elapsed1/elapsed2))
```

```
print("Try a larger n to see an even greater savings!")
```

Using a list... count= 250 and time = 2.5040 seconds

Using a set.... count= 250 and time = 0.1520 seconds

With n=500, sets ran about 16.5 times faster than lists!

Try a larger n to see an even greater savings!

Các Hàm và Toán Tử Trên Set

- **Kích thước của set: len()**

```
s = { 2, 3, 2, 4, 3 }  
print(len(s)) # 3
```

- **Copying set : copy()**

```
s = { 1, 2, 3 }  
t = s.copy()  
s.add(4)  
print(s)  
print(t)
```

```
set({1, 2, 3, 4})  
set({1, 2, 3})
```

- **Xóa set : clear()**

```
s = { 1, 2, 3 }  
s.clear()  
print(s, len(s)) # set() 0
```

Các Hàm và Toán Tử Trên Set

- ❖ Các phép toán trên set và phần tử trong set

- **Toán tử: in and not in**

```
s = { 1, 2, 3 }  
print(0 in s)    #False  
print(1 in s)    #True
```

```
s = { 1, 2, 3 }  
print(0 not in s) #True  
print(1 not in s) #False
```

- **Thêm phần tử: add()**

```
s = { 1, 2, 3 }  
print(s, 4 in s) # set({1, 2, 3}) False  
s.add(4)  
print(s, 4 in s) # set({1, 2, 3, 4}) True
```

Các Hàm và Toán Tử Trên Set

- **Xóa phần tử: remove() vs. discard()**

```
s = { 1, 2, 3 }  
print(s, 3 in s) # set({1, 2, 3}) True  
s.remove(3)  
print(s, 3 in s) # set({1, 2}) False  
s.remove(3) # crashes; ERROR
```

```
s = { 1, 2, 3 }  
print(s, 3 in s) # set({1, 2, 3}) True  
s.discard(3)  
print(s, 3 in s) # set({1, 2}) False  
s.discard(3) # does not crash! No ERROR  
print(s, 3 in s) # set({1, 2}) False
```

Các Hàm và Toán Tử Trên Set

- Các phép toán trên hai sets

a) $s.\text{issubset}(t)$ tương đương $s \leq t$

```
print({1,2} <= {1}, {1,2}.issubset({1}))  
print({1,2} <= {1,2}, {1,2}.issubset({1,2}))  
print({1,2} <= {1,2,3}, {1,2}.issubset({1,2,3}))
```

```
False False  
True True  
True True
```

Các Hàm và Toán Tử Trên Set

- Các phép toán trên hai sets

b) s.**issuperset**(t) tương đương $s \supseteq t$

```
print({1,2} >= {1}, {1,2}.issuperset({1}))  
print({1,2} >= {1,2}, {1,2}.issuperset({1,2}))  
print({1,2} >= {1,2,3}, {1,2}.issuperset({1,2,3}))
```

```
True True  
True True  
False False
```

Các Hàm và Toán Tử Trên Set

- Các phép toán trên hai sets

c) $s.\text{union}(t)$ tương đương $s \mid t$

```
print({1,2} | {1}, {1,2}.union({1}))  
print({1,2} | {1,3}, {1,2}.union({1,3}))  
s = {1,2}  
t = s | {1,3}  
print(s, t)
```

```
set({1, 2}) set({1, 2})  
set({1, 2, 3}) set({1, 2, 3})  
set({1, 2}) set({1, 2, 3})
```

Các Hàm và Toán Tử Trên Set

- Các phép toán trên hai sets

d) `s.intersection(t)` tương đương `s & t`

```
print({1,2} & {1}, {1,2}.intersection({1}))  
print({1,2} & {1,3}, {1,2}.intersection({1,3}))  
s = {1,2}  
t = s & {1,3}  
print(s, t)
```

```
set({1}) set({1})  
set({1}) set({1})  
set({1, 2}) set({1})
```

Các Hàm và Toán Tử Trên Set

- Các phép toán trên hai sets

e) s.**difference**(t) tương đương $s - t$

```
print({1,2} - {1}, {1,2}.difference({1}))  
print({1,2} - {1,3}, {1,2}.difference({1,3}))  
s = {1,2}  
t = s - {1,3}  
print(s, t)
```

```
set({2}) set({2})  
set({2}) set({2})  
set({1, 2}) set({2})
```

Các Hàm và Toán Tử Trên Set

- Các phép toán trên hai sets

f) s. **symmetric_difference**(t) tương đương $s \Delta t$

```
# Khởi tạo A và B
```

```
A = {1, 2, 3, 4, 5}
```

```
B = {4, 5, 6, 7, 8}
```

```
# Sử dụng toán tử ^
```

```
# Output: {1, 2, 3, 6, 7, 8}
```

```
print(A ^ B)
```

```
# Sử dụng symmetric_difference() trên A
```

```
# Output: {1, 2, 3, 6, 7, 8}
```

```
print(A.symmetric_difference(B))
```

Các Hàm và Toán Tử Trên Set

- Các phép toán trên hai sets

g) `s.update(t)` tương đương `s |= t`

(Cập nhật set với hợp của chính nó và set khác.)

```
s = {1, 2}
t = {1, 3}
u = {2, 3}
s.update(u)
t |= u
print(s, t, u)
```

```
set({1, 2, 3}) set({1, 3, 2}) set({2, 3})
```

Các Hàm và Toán Tử Trên Set

- * Lưu ý: Set chứa các phần tử không theo thứ tự

```
s1 = [1, 2, 3]
s2 = [3, 2, 1]
print(s1==s2)  # So Sánh 2 List : False
```

```
t1 = {1,2,3}
t2 = {3,2,1}
print(t1==t2)  # So Sánh 2 Set: True
```

Dictionary

Nội Dung

- Cách tạo Dictionary
- Các hàm và toán tử trên Dictionary

Cách Tạo Dictionary

- Từ điển (Dictionary) được dùng để ánh xạ hoặc liên kết dữ liệu bạn cần lưu trữ (**value**) và khóa (**key**) bạn cần để lấy ra dữ liệu đó.
- Từ điển trong Python được định nghĩa gồm hai thành phần là khóa (key) và giá trị (value).
 - Khóa là đối tượng mang tính duy nhất.
 - Giá trị được lưu có thể là 1D List hoặc 2D List, một chuỗi, một số, một đối tượng bất kỳ trong python ...

```
Dict = {'Name': 'Tim', 'Age': 18, ...}
```

Cách Tạo Dictionary

- **Tạo một từ điển rỗng:** dùng `dict()` hoặc `{ }`

```
d = dict()
```

```
print(d) # prints {}
```

```
d = { }
```

```
print(d) # prints {}
```

- **Tạo từ điển từ một list các cặp (key, value)**

```
pairs = [("cow", 5), ("dog", 98), ("cat", 1)]
```

```
d = dict(pairs)
```

```
print(d)
```

```
{ 'cow': 5, 'dog': 98, 'cat': 1 }
```

Cách Tạo Dictionary

- Tạo từ điển bằng cặp phát tĩnh

```
d = { "cow":5, "dog":98, "cat":1 }  
print(d)
```

```
{'cow': 5, 'dog': 98, 'cat': 1}
```

- Các ví dụ khác:

```
d1 = {1: 'Quantrimang.com', 2: 'Công nghệ'}
```

```
d2 = {'tên': 'QTM', 1: [1, 3, 5]}
```

```
d3 = dict({1:'apple', 2:'ball'})
```

```
d4 = dict([(1,'QTM'), (2,'CN')])
```

Cách Tạo Dictionary

- ❑ Các tính chất của từ điển:
 - Cần lưu ý khi sử dụng khóa trong từ điển:
 - Một khóa không thể xuất hiện hai lần (khóa không được trùng nhau).
 - Giá trị được lưu trong từ điển có thể thuộc bất kỳ kiểu nào trong khi khóa phải là kiểu bất biến như số, tuple hoặc chuỗi.
 - Khóa sử dụng trong từ điển có phân biệt chữ hoa chữ thường - Cùng tên khóa nhưng tên khóa viết hoa và viết thường sẽ được coi là các khóa khác nhau.

Cách Tạo Dictionary

- Các tính chất của từ điển:
 - Từ điển ánh xạ một khóa đến một giá trị

```
ages = dict()
key = "tom"
value = 38
ages[key] = value # "tom" is the key, 38 is the value
print(ages)
print(ages[key])
```

```
{'tom': 38}
```

```
38
```

Cách Tạo Dictionary

- Các từ khóa biểu diễn theo kiểu tập hợp
 - Không có thứ tự

```
d = dict()  
d[2] = 100  
d[4] = 200  
d[8] = 300
```

```
{2: 100, 4: 200, 8: 300}
```

```
print(d) # unpredictable order
```

- Duy nhất

```
d = dict()  
d[2] = 100  
d[2] = 200  
d[2] = 400
```

```
{2: 400}
```

```
print(d) # { 2:400 }
```

Cách Tạo Dictionary

● Các từ khóa không thể thay đổi

```
d = dict()
a = [1] # lists are mutable, so...
d[a] = 42 # Error: unhashable type: 'list'
```

● Các giá trị có thể thay đổi

```
d = dict()
a = [1,2]
d["fred"] = a
print(d)
print(d["fred"])
a += [3]
print(d["fred"])
# but keys may not be mutable
```

```
d[a] = 42 # TypeError: unhashable type: 'list'
```

```
{'fred': [1, 2]}
```

```
[1, 2]
```

```
[1, 2, 3]
```

```
Traceback (most recent call last):
```

```
TypeError: unhashable type: 'list'
```

Các Hàm và Toán Tử Trên Dictionary

❑ Các phép toán trên từ điển

▪ Tìm chiều dài: `len()`

```
d = { 1:[1,2,3,4,5], 2:"abcd" }  
print(len(d))  # 2
```

▪ Tạo bản sao : `copy()`

```
d1 = { 1:"a" }  
d2 = d1.copy()  
d1[2] = "b"  
print(d1)  
print(d2)
```

```
{1: 'a', 2: 'b'}  
{1: 'a'}
```

Các Hàm và Toán Tử Trên Dictionary

- Xóa tất cả phần tử trong từ điển: `clear()`

```
d = { 1: "a", 2: "b" }  
d.clear()  
print(d, len(d))
```

```
{ } 0
```

- Vòng lặp trên từ điển:

```
d = { 1: "a", 2: "b" }  
for key in d:  
    print(key, d[key])
```

```
1 a  
2 b
```

Các Hàm và Toán Tử Trên Dictionary

- ❑ Các phép toán trên từ điển và khóa

- Toán tử in và not in:

```
d = { 1:"a", 2:"b" }  
print(0 in d) # False  
print(1 in d) # True  
print("a" in d) # surprised? False
```

```
d = { 1:"a", 2:"b" }  
print(0 not in d) # True  
print(1 not in d) # False  
print("a" not in d) # True
```

Các Hàm và Toán Tử Trên Dictionary

- **Toán tử [key]** : trả về giá trị tương ứng với khóa

```
d = { 1:"a", 2:"b" }  
print(d[1]) # a  
print(d[3]) # crash!
```

- **Toán tử gán =** : gán giá trị cho từ khóa tương ứng

```
d = { 1:"a", 2:"b" }  
print(d[1]) # a  
d[1] = 42  
print(d[1]) # 42
```

Các Hàm và Toán Tử Trên Dictionary

- **Hàm `get()`:** `get(key, default)` trả về giá trị tương ứng với key hoặc nếu không tồn tại trả về default (`None` nếu default không được dùng)

```
d = { 1:"a", 2:"b" }  
print(d.get(1)) # tương đương d[1]  
print(d.get(1, 42)) # default không được dùng  
print(d.get(0)) # doesn't crash! Không bị lỗi  
print(d.get(0, 42)) # default được dùng
```

a

a

None

42

Các Hàm và Toán Tử Trên Dictionary

- **Toán tử del:** xóa một từ khóa khỏi từ điển

```
d = { 1:"a", 2:"b" }  
print(1 in d)    # True  
del d[1]  
print(1 in d)    # False  
del d[1] # crash! ERROR
```

Các Hàm và Toán Tử Trên Dictionary

- Thay đổi giá trị và thêm phần tử cho từ điển

```
d2 = {1: 'Quantrimang.com', 'quantrimang': 'Công nghệ'}
```

```
d2['quantrimang'] = 'Quản trị mạng'
```

```
#output: {1: 'Quantrimang.com', 'quantrimang': 'Quản trị mạng'}
```

```
print(d2)
```

```
d2[2] = 'Python'
```

```
#output: {1: 'Quantrimang.com', 'quantrimang': 'Quản trị mạng', 2: 'Python'}
```

```
print(d2)
```

Các Hàm và Toán Tử Trên Dictionary

- Trả về danh sách đối tượng dùng **items()** hoặc từ khóa dùng **keys()**

```
d = { 1:"a", 2:"b" }  
print(d)  
print(list(d.items())) # [(1, 'a'), (2, 'b')]  
print(list(d.keys())) # [1, 2]
```

```
{1: 'a', 2: 'b'}  
[(1, 'a'), (2, 'b')]  
[1, 2]
```

Các Hàm và Toán Tử Trên Dictionary

- Sắp xếp các từ khóa trong từ điển

```
Dict = {'Tim': 18, 'Charlie':12, 'Tiffany':22, 'Robert':25}
Students = list(Dict.keys())
Students.sort()
print(Students) # ['Charlie', 'Robert', 'Tiffany', 'Tim']
for S in Students:
    print(":".join((S,str(Dict[S]))))
```

```
['Charlie', 'Robert', 'Tiffany', 'Tim']
Charlie:12
Robert:25
Tiffany:22
Tim:18
```

Các Hàm và Toán Tử Trên Dictionary

- Cập nhật từ điển dùng: `update()`

```
d1 = { 1: "a", 2: "b" }
```

```
d2 = { 2: "c", 3: "d" }
```

```
d1.update(d2)
```

```
d2[4] = "e"
```

```
print(d1)
```

```
print(d2)
```

```
{1: 'a', 2: 'c', 3: 'd'}
```

```
{2: 'c', 3: 'd', 4: 'e'}
```

Các Hàm và Toán Tử Trên Dictionary

- **Dictionary comprehension**

```
lap_phuong = {x: x*x*x for x in range(6)}  
# Output: {0: 0, 1: 1, 2: 8, 3: 27, 4: 64, 5: 125}  
print(lap_phuong)
```

```
lap_phuong_chan = {x: x*x*x for x in range (10) if x%2==0}  
# output: {0: 0, 2: 8, 4: 64, 6: 216, 8: 512}  
print(lap_phuong_chan)
```

Chuyển Đổi Giữa Các Kiểu Dữ Liệu

```
print(float(11)) # 11.0
```

```
print(int(18.6)) # 18
```

```
print(set([2,4,6])) # set({2, 4, 6})
```

```
print(tuple({3,5,7})) # (3, 5, 7)
```

```
print(list('hello')) # ['h', 'e', 'l', 'l', 'o']
```

```
print(dict([[2,4],[1,3]])) # {1: 3, 2: 4}
```

```
print(dict([(3,9),(4,16)])) # {3: 9, 4: 16}
```

Câu hỏi

Bài 1. Làm sạch tên trạm

```
stations = [  
    "Tan Son Hoa",  
    "Thu Duc",  
    "Tan Son Hoa",  
    "Bien Hoa",  
    "Thu Duc"  
]
```

- Loại bỏ tên trạm trùng lặp.
- In số lượng trạm duy nhất.

Câu hỏi

- Cho dữ liệu PM2.5 tại các trạm:

```
data = {  
  "Tan Son Hoa": [18.2, 22.5, 35.1, 28.4],  
  "Thu Duc": [15.3, 17.8, 19.0, 21.6],  
  "Bien Hoa": [25.4, 30.2, 33.8, 29.1]  
}
```

Yêu cầu:

- Tính PM2.5 trung bình tại mỗi trạm.
- Xác định trạm có PM2.5 trung bình cao nhất.
- Đếm số lần đo vượt ngưỡng 25 tại từng trạm.

Unpacking operator

- Unpacking là thao tác tách các phần tử từ iterable hoặc dictionary để gán vào biến, truyền vào hàm hoặc kết hợp dữ liệu

```
coordinates = (10.8231, 106.6297)
x, y = coordinates

print(x)
print(y)
```

Unpacking list và tuple

```
first, second, third = [10, 20, 30]  
print(first, second, third)
```

```
station, pm25, temperature = ("Tan Son Hoa", 31.2, 29.5)
```

Lưu ý

- Số biến nhận phải tương ứng với số phần tử, nếu không sẽ phát sinh lỗi.
- Có thể dùng `_` để bỏ qua một giá trị không cần sử dụng.

```
name, _, pm25 = ["A01", "unused", 28.5]
```

Extended unpacking với *

```
values = [10, 20, 30, 40, 50]
```

```
first, *middle, last = values
```

```
print(first) # 10
```

```
print(middle) # [20, 30, 40]
```

```
print(last) # 50
```

- **Ứng dụng**

Tách phần đầu, phần cuối và phần giữa.

Làm việc với dữ liệu có số lượng phần tử biến đổi

Unpacking khi gọi hàm

```
def calculate_aqi(pm25, pm10, no2):  
    return pm25 + pm10 + no2  
  
values = [25.3, 40.2, 15.6]  
  
result = calculate_aqi(*values)  
print(result)
```

- **Giải thích**
- *values giải nén list/tuple thành các đối số vị trí.
- Tương đương: `calculate_aqi(values[0], values[1], values[2])`

*args vs **kwargs

	*args	**kwargs
Ý nghĩa	Viết tắt của arguments (đối số).	Viết tắt của keyword arguments (đối số từ khóa).
Kiểu dữ liệu	tuple (bộ giá trị).	dict (từ điển gồm key-value).
Cách truyền	Truyền các giá trị cách nhau bằng dấu phẩy: func(1, 2, 3).	Truyền theo cặp tên và giá trị: func(name="An", age=20).
Mục đích	Cho phép hàm nhận số lượng tham số vị trí tùy ý.	Cho phép hàm nhận số lượng tham số từ khóa tùy ý.

*args

```
def calculate_mean(*values):  
    return sum(values) / len(values)  
  
print(calculate_mean(10, 20, 30, 40))
```

Ý nghĩa

- *args thu thập nhiều đối số vị trí.
- Bên trong hàm, args là tuple.
- Tên args có thể thay đổi, nhưng dấu * là bắt buộc.

****kwargs**

Ý nghĩa

- ****kwargs** thu thập các đối số có tên.
- Bên trong hàm, **kwargs** là dictionary.
- Hữu ích khi số lượng thuộc tính chưa biết trước.

```
def print_station_info(**kwargs):  
    for key, value in kwargs.items():  
        print(f"{key}: {value}")  
  
print_station_info(  
    station_name="Tan Son Hoa",  
    pm25=32.5,  
    temperature=29.0  
)
```

Unpacking dictionary

- The **double asterisk operator** (**) is the primary syntax used to unpack a dictionary's key-value pairs

```
user_profile = {"name": "Alice", "age": 25}
user_settings = {"theme": "dark", "age": 26}

# Merge dictionaries
combined = {**user_profile, **user_settings}

print(combined)
# Output: {'name': 'Alice', 'age': 26, 'theme': 'dark'}
```

Nếu key trùng nhau, giá trị xuất hiện phía sau sẽ ghi đè giá trị phía trước.

```
station = {
    "station_name": "Tan Son Hoa",
    "pm25": 32.5,
    "temperature": 29.0
}

def report(station_name, pm25, temperature):
    print(f"Trạm: {station_name}")
    print(f"PM2.5: {pm25}")
    print(f"Nhiệt độ: {temperature}")

report(**station)
```

Câu hỏi

- Viết hàm: `summarize_measurements(*values)`

Hàm trả về dictionary gồm:

- `min`
- `max`
- `mean`
- `count`

Ví dụ:

```
print(summarize_measurements(10, 20, 30, 40))
```

```
# Kết quả: {'min': 10, 'max': 40, 'mean': 25.0, 'count': 4}
```

```
print(summarize_measurements())
```

```
# Kết quả: {'min': None, 'max': None, 'mean': None, 'count': 0}
```

Generator expression

- **Generator expression** (biểu thức sinh) trong Python là một cấu trúc cú pháp dùng để tạo ra một generator ẩn danh một cách nhanh chóng
- **Đặc điểm chính**
 - **Cú pháp:** Giống với List Comprehension, nhưng dùng dấu ngoặc tròn () thay vì dấu ngoặc vuông [].
 - **Tiết kiệm bộ nhớ:** Không lưu toàn bộ các phần tử vào RAM cùng lúc. Nó chỉ tính toán và trả về từng giá trị một (lazy evaluation) khi được gọi tới.
 - **Chỉ lặp được một lần:** Sau khi duyệt qua hết các phần tử, generator expression sẽ cạn kiệt và không lặp lại được nữa

Generator expression

```
values = [10, 20, 30, 40, 50]

squares_generator = (x ** 2 for x in values)

print(squares_generator)
print(next(squares_generator))
print(next(squares_generator))
```

So sánh list và generator

Tiêu chí	List comprehension	Generator expression
Ký hiệu	[. . .]	(. . .)
Tạo dữ liệu	Ngay lập tức	Theo nhu cầu
Bộ nhớ	Cao hơn nếu dữ liệu lớn	Thấp hơn
Có thể truy cập lại	Có	Thường chỉ duyệt một lần
Phù hợp	Dữ liệu nhỏ/vừa, cần tái sử dụng	Dòng dữ liệu lớn, tính toán tuần tự

Bài tập:

- Bài toán: Báo cáo nhanh chất lượng không khí

```
measurements = [  
    ("TSH01", "Tan Son Hoa", 32.5),  
    ("TD02", "Thu Duc", 18.2),  
    ("BH03", "Bien Hoa", 41.6),  
    ("NB04", "Nha Be", 25.0),  
    ("TSH01", "Tan Son Hoa", 29.8)  
]
```

Yêu cầu:

1. Dùng unpacking để tách từng bản ghi.
2. Tạo set tên các trạm không trùng lặp.
3. Dùng list comprehension lấy các phép đo PM2.5 > 25.
4. Tạo dictionary `station_id: pm25`.
5. Dùng generator expression để tính tổng PM2.5 của các giá trị vượt ngưỡng.
6. Viết hàm `summarize(*values)` trả về min, max và mean.